Agile Software Development

=>Agile methods:

.Why agile.

Agile was created to overcome the challenges of old software design methods, which were slow and inflexible. Agile methods focus more on creating code quickly rather than spending too much time on design.

.Goals:

Agile aims to make software development faster and adaptable by reducing the need for excessive documentation and responding quickly to any requirement changes.

=> Agile manifesto (strategy / platform).

. Core values:

The agile manifesto outlines values that prioritize:

. Individuals and interactions over processes.

. Working software over comprehensive documentation.

. Customer collaboration over contract negotiation.

. Responding to change over following a plan.

=> Rapid software development:

. Need for speed:

Businesses need software that can adapt quickly. Agile encourages developing and updating software rapidly to keep up with changing business needs.

.Iterative process:

Agile uses an iterative approach where development, design, and testing are done in short cycles, and stakeholders review each version.

=> Agile method applicability:

.Where to use agile:

Agile works best for small to medium projects where the team is involved in frequent updates and collaboration with customers.

.Challenges:

It may not scale well to large projects due to informal processes, lack of documentation, and need for close teamwork.

=> Problems with agile methods:

.Customer engagement:

keeping the customer engaged can be tough, especially when many stakeholders are involved.

- **Prioritizing changes:**
Handling changes is tricky when different stakeholders have conflicting priorities.
- **Keeping it simple:**
Ensuring simplicity requires effort, and team may struggle with Agile's less formal, team-defined processes.

=> Agile methods and software maintance:

- **Supporting maintenance:**
Since most budgets go towards maintaing software, agile must be adaptable for both new development and maintenance.

- **Documentation concerns:**
Agile projects often lack formal documentation, which can make future maintenance harder.

⇒ Technical, human and organizational issues:

Some projects need careful planning and detailed specifications, while others benefit from Agile's faster, flexible approach. The size of the project, team structure, system type, and regulations influence this balance

⇒ Extreme programming:

XP is a well-known agile method with frequent code updates, customer involvement, and tests that ensure each feature works before integrating it.

⇒ XP and agile principles:

. Incremental development:

New versions are regularly released.

. Teamwork and simplicity:

Programmers work in pairs, share code ownership and keep code simple to avoid unecessary complications.

.Customer involvement:

Customers actively participate and help define tests to ensure the software meets their needs.

⇒Refactoring:

Programmers make small improvements to the code even without immediate issues, making it simpler and easier to maintain in the future. This practice is key in agile to keep code clean and understandable.

⇒Testing in XP:

-Test-first development:

In XP, tests are written before code, clarifying what the software should do. Automated testing ensures that every new change does not break existing functionality.

The extreme programming release cycle

⇒Pair programming:

Two programmers work together at one workstation, reviewing each other's code. The approach reduces errors, promotes shared knowledge, and mitigates risks when team members leave.

⇒Scrum:

Scrum is an agile project management method focused on quick, iterative development cycles called sprints. Each sprint delivers part of the project, with team member continuously tracking progress.

⇒Scrum roles:

.Product owner:

Represents the business, prioritizes tasks.

.Scrum master:

keeps the team on track, shields them from distractions.

- **Development team:**
  Responsible for completing tasks within the sprint.

⇒ Scaling agile methods:

- **Challenges with scaling:**
  Agile is easier for smaller projects, scaling it to larger projects with distributed teams requires careful planning and documentation.

⇒ Scaling out and scaling up:

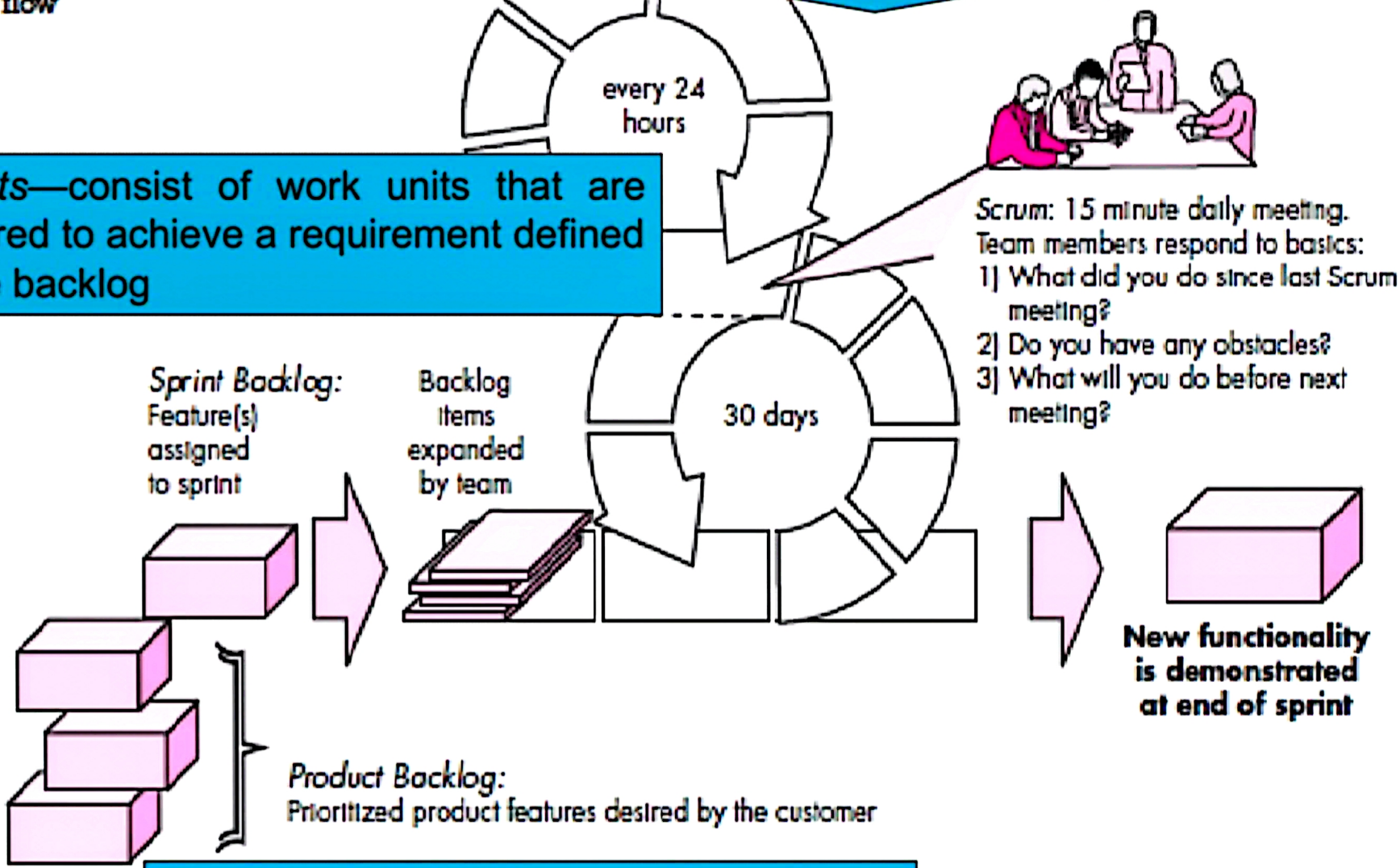
- **Scaling up:**
  Adapting agile for large, complex projects.

- **Scaling out:**
  Implementing agile across large organizations.

Scrum process flow

This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them

Sprints—consist of work units that are required to achieve a requirement defined in the backlog



list of work to be done on the project.

⇒ Key takeaways:

- **Testing importance:**
  Automated tests are essential in agile, especially in XP, to ensure quality with every update.
- **Scrum's strength:**
  Scrum effectively manages projects with sprints, but agile's flexibility makes it challenging for large systems needing more structure.